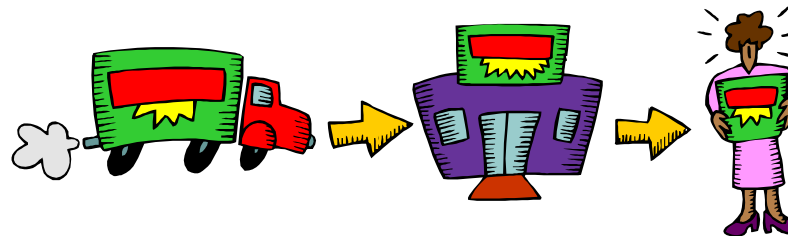


Công nghệ phần mềm

Cài đặt phần mềm (Implementation)



Nội dung

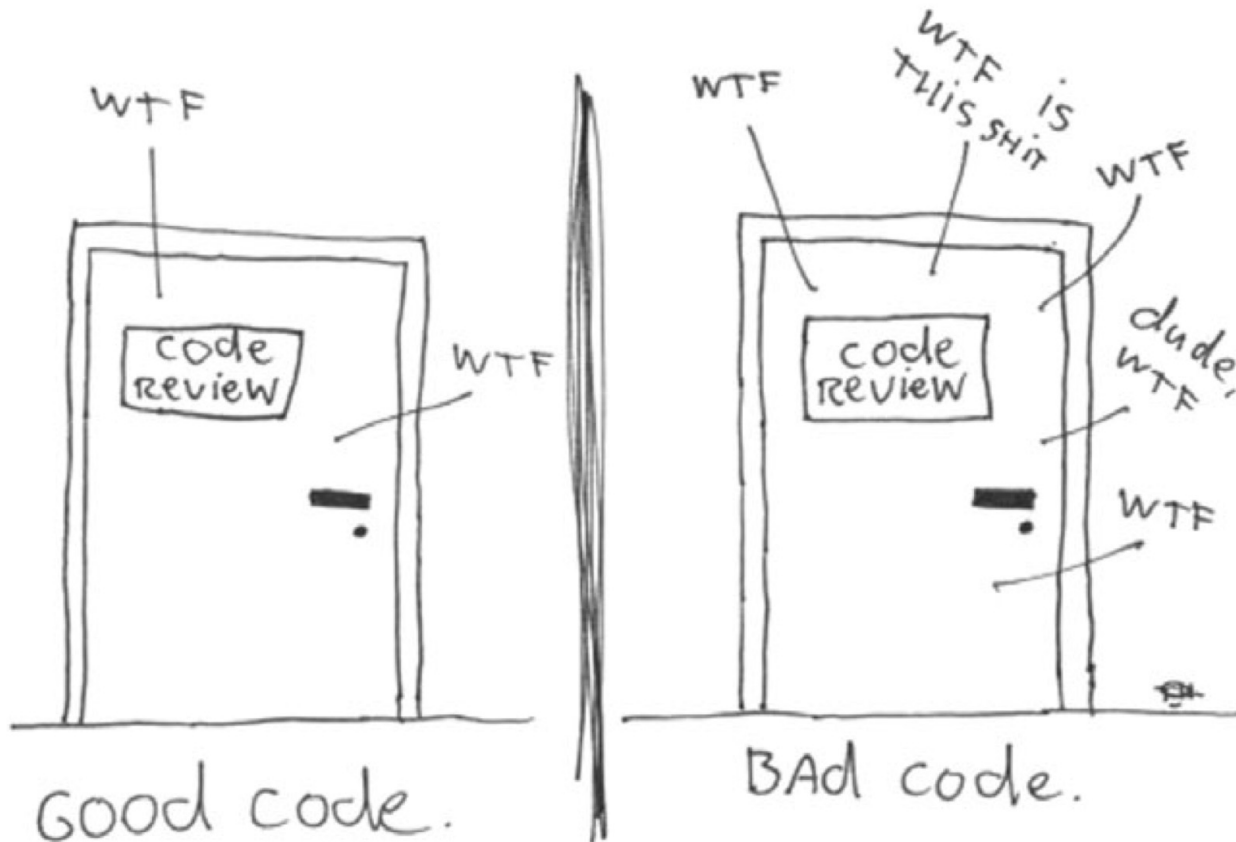
- Giới thiệu
- Thế nào là “Clean code”
- Phong cách lập trình
- Tái sử dụng mã nguồn
- Quản lý các phiên bản mã nguồn
- Gỡ lỗi

Giới thiệu

- Nhớ lại 5 hoạt động chính trong quy trình PTPM nói chung
 - Communication ✓
 - Planning ✓
 - Modeling ✓
 - Construction = Implementation + Testing
 - Deployment

“CLEAN CODE” LÀ GÌ?

The ONLY VALID MEASUREMENT OF CODE QUALITY: WTFs/MINUTE



(c) 2008 Focus Shift

Một vài kiến giải ...

- Mã nguồn cần đơn giản, ít phụ thuộc, xử lý ngoại lệ tốt, và có hiệu suất tốt

**Bjarne Stroustrup, inventor of C++
and author of *The C++ Programming Language***

I like my code to be elegant and efficient. The logic should be straightforward to make it hard for bugs to hide, the dependencies minimal to ease maintenance, error handling complete according to an articulated strategy, and performance close to optimal so as not to tempt people to make the code messy with unprincipled optimizations. Clean code does one thing well.

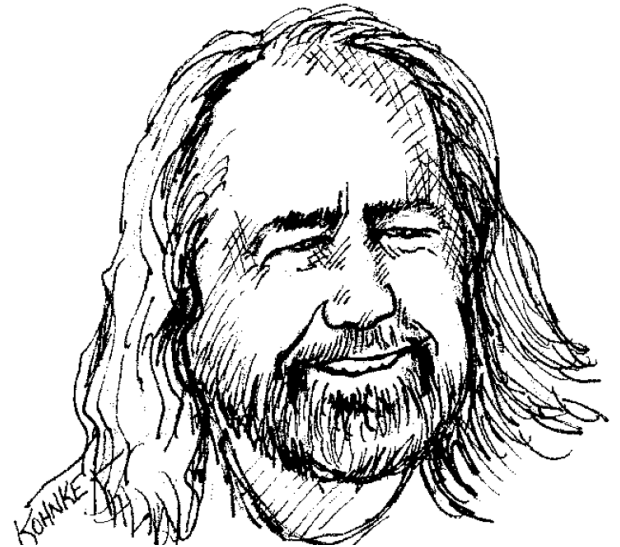


Một vài kiến giải ...

- Đọc code giống như đọc một bài văn xuôi, một câu chuyện, v.v. được viết rõ ràng và lời cuốn
- Mã nguồn chỉ chứa những thông tin cần thiết

Grady Booch, author of *Object Oriented Analysis and Design with Applications*

Clean code is simple and direct. Clean code reads like well-written prose. Clean code never obscures the designer's intent but rather is full of crisp abstractions and straightforward lines of control.

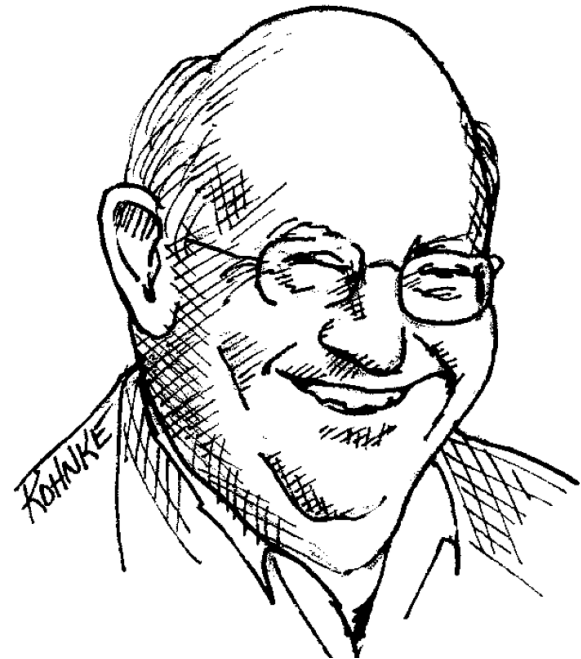


Một vài kiến giải ...

Mã nguồn phải được **kiểm thử cẩn thận**, có phụ thuộc nhỏ nhất, dễ đọc

“Big” Dave Thomas, founder of OTI, godfather of the Eclipse strategy

Clean code can be read, and enhanced by a developer other than its original author. It has unit and acceptance tests. It has meaningful names. It provides one way rather than many ways for doing one thing. It has minimal dependencies, which are explicitly defined, and provides a clear and minimal API. Code should be literate since depending on the language, not all necessary information can be expressed clearly in code alone.



Tự kiến giải

- Bạn mong muốn mã nguồn như thế nào khi bạn là
 - Nhà phát triển
 - Kiểm thử viên
 - Người bảo trì sản phẩm

PHONG CÁCH LẬP TRÌNH

Đặt tên biến, tên hàm, tên lớp

- Tên phải có nghĩa, gợi nhớ
 - Sử dụng các ký hiệu, từ Tiếng anh có nghĩa
 - Làm cho dễ đọc, dễ hiểu
 - Thí dụ: `DateOfBirth` hoặc `date_of_birth`
 - Không viết `dateofbirth`
- Tránh đặt tên quá dài
- Một vài quy chuẩn
 - Tên lớp bắt đầu bằng chữ hoa
 - Tên hằng số toàn chữ hoa
 - Biến vòng lặp, chỉ số: `i` (`iteration`, `index`)

Tìm điểm chung hai đoạn mã nguồn

```
public List<int[]> getThem() {  
    List<int[]> list1 = new ArrayList<int[]>();  
    for (int[] x : theList)  
        if (x[0] == 4)  
            list1.add(x);  
    return list1;  
}
```

```
public List<int[]> getFlaggedCells() {  
    List<int[]> flaggedCells = new ArrayList<int[]>();  
    for (int[] cell : gameBoard)  
        if (cell[STATUS_VALUE] == FLAGGED)  
            flaggedCells.add(cell);  
    return flaggedCells;  
}
```

Câu lệnh

- Câu lệnh phải mô tả cấu trúc
 - Tụt lề, dễ đọc, dễ hiểu
- Làm đơn giản các câu lệnh
 - Mỗi lệnh trên 1 dòng
 - Triển khai các biểu thức phức tạp
 - Hạn chế truyền tham số là kết quả của hàm, biểu thức:
- Tránh các cấu trúc phức tạp
 - Các lệnh if lồng nhau
 - Điều kiện phủ định if not

Ví dụ

Trên 1 dòng

```
if (g < 17 && h < 22 || i < 60) { return true; } else {System.out.println ("incorrect") ; return false; }
```

Thay bằng

```
if (g < 17 && h < 22 || i < 60) {  
    return true;  
} else {  
    System.out.println("incorrect");  
    return false;  
}
```

Chú thích

- Chú thích rất quan trọng: hỗ trợ đáng kể tính dễ đọc và dễ bảo trì của mã nguồn
- Cách viết chú thích
 - Biến: mục đích sử dụng của các biến, lưu ý khi dùng biến, v.v.
 - Hàm: Mục đích hàm, giá trị truyền vào, giá trị trả về, lưu ý khi dùng hàm, v.v.
 - Class: Mục đích, tác giả, giấy phép

Chú thích



*“Don’t **comment** bad code—rewrite it.”*

—Brian W. Kernighan and P. J. Plaugher¹

Ví dụ (good comment)

- Mã nguồn String.java

```
public final class String
    implements java.io.Serializable, Comparable<String>, CharSequence,
               Constable, ConstantDesc {

    /**
     * The value is used for character storage.
     *
     * @implNote This field is trusted by the VM, and is a subject to
     * constant folding if String instance is constant. Overwriting this
     * field after construction will cause problems.
     *
     * Additionally, it is marked with {@link Stable} to trust the contents
     * of the array. No other facility in JDK provides this functionality (yet).
     * {@link Stable} is safe here, because value is never null.
     */
    @Stable
    private final byte[] value;

    /**
     * The identifier of the encoding used to encode the bytes in
     * {@code value}. The supported values in this implementation are
     *
     * LATIN1
     * UTF16
     *
     * @implNote This field is trusted by the VM, and is a subject to
     * constant folding if String instance is constant. Overwriting this
     * field after construction will cause problems.
     */
    private final byte coder;

    /** Cache the hash code for the string */
    private int hash; // Default to 0
}
```

Ví dụ (good comment)

- Làm rõ hơn những nội dung khó hiểu bằng ngôn ngữ tự nhiên

```
// format matched kk:mm:ss EEE, MMM dd, yyyy  
Pattern timeMatcher = Pattern.compile(  
    "\\d*:\\d*:\\d* \\w*, \\w* \\d*, \\d*");
```

Ví dụ (good comment)

- Đoạn code có rủi ro được cảnh báo (not thread safe)

```
public static SimpleDateFormat makeStandardHttpDateFormat()
{
    //SimpleDateFormat is not thread safe,
    //so we need to create each instance independently.
    SimpleDateFormat df = new SimpleDateFormat("EEE, dd MMM yyyy HH:mm:ss z");
    df.setTimeZone(TimeZone.getTimeZone("GMT"));
    return df;
}
```

Ví dụ (good comment)

- Dùng TODO để đánh dấu những việc cần phải làm sớm (thời điểm hiện tại chưa làm được)
- Hầu hết các IDEs hỗ trợ định vị comment TODO

```
//TODO-MdM these are not needed
// We expect this to go away when we do the checkout model
protected VersionInfo makeVersion() throws Exception
{
    return null;
}
```

Ví dụ (bad comment)

- Comment vô nghĩa vì đoạn code rất dễ để hiểu

waitForClose

```
// Utility method that returns when this.closed is true. Throws an exception
// if the timeout is reached.
public synchronized void waitForClose(final long timeoutMillis)
throws Exception
{
    if(!closed)
    {
        wait(timeoutMillis);
        if(!closed)
            throw new Exception("MockResponseSender could not be closed");
    }
}
```

Ví dụ (bad comment)

- Tên biến và comment có nội dung giống nhau

```
/** The day of the month. */  
private int dayOfMonth;  
  
/** The name. */  
private String name;  
  
/** The version. */  
private String version;  
  
/** The licenceName. */  
private String licenceName;  
  
/** The version. */  
private String info;
```

Ví dụ (bad comment)

- Comment thứ hai vô nghĩa

startSending

```
private void startSending()
{
    try
    {
        doSending();
    }
    catch(SocketException e)
    {
        // normal. someone stopped the request.
    }
    catch(Exception e)
    {
        try
        {
            response.add(Responder.makeExceptionString(e));
            response.closeAll();
        }
        catch(Exception e1)
        {
            //Give me a break!
        }
    }
}
```

TÁI SỬ DỤNG MÃ NGUỒN

Tái sử dụng mã nguồn

- Tái sử dụng các thành phần phần mềm (components) định nghĩa trước
 - Tránh trường hợp “Phát minh lại bánh xe”
 - Sử dụng những đoạn mã đã được thẩm định chất lượng

Viết mã nguồn để tái sử dụng

- Đặc điểm của phần mềm để tái sử dụng
 - Phân chia mô-đun
 - Có tính đóng gói
- Viết mã nguồn để tái sử dụng:
 - Tạo thư viện phần mềm
 - Lập trình chung (generic programming)
 - Phần mềm sinh mã tự động (generators)

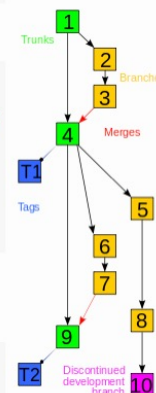
QUẢN LÝ CÁC PHIÊN BẢN MÃ NGUỒN

Quản lý các phiên bản mã nguồn

- Quản lý quá trình chỉnh sửa mã nguồn của một nhóm phát triển phần mềm
- Các khái niệm cơ bản
 - Repository (kho lưu trữ)
 - Lưu trữ dự án chung của đội phát triển phần mềm
 - Working copies
 - Check out: Lấy dự án từ server về máy cục bộ
 - Commit: Đưa các thay đổi lên server
 - Update: Cập nhật thay đổi từ thành viên khác về máy cục bộ
 - Merge: Nhiều thành viên cập nhật trên một tệp tin
 - V..v.

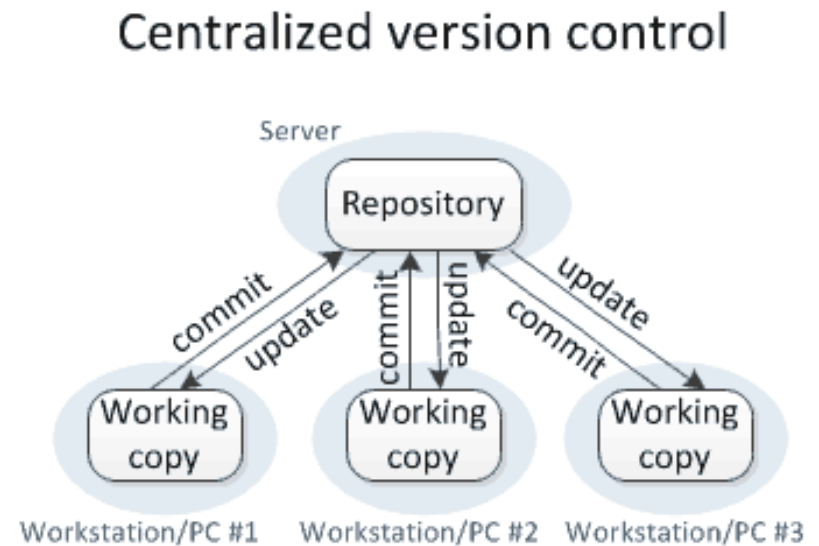
Quản lý các phiên bản mã nguồn

V·T·E Version control software [hide]		
Years, where available, indicate the date of first stable release. Systems with names <i>in italics</i> are no longer maintained or have planned end-of-life dates.		
Local only	Free/open-source	RCS (1982) · SCCS (1972)
	Proprietary	PVCS (1985) · QVCS (1991)
Client–server	Free/open-source	CVS (1986, 1990 in C) · CVSNT (1998) · QVCS Enterprise (1998) · Subversion (2000)
	Proprietary	AccuRev SCM (2002) · Azure DevOps (Server (via TFVC) (2005) · Services (via TFVC) (2014)) · ClearCase (1992) · CMVC (1994) · Dimensions CM (1980s) · DSEE (1984) · Endavor (1980s) · Integrity (2001) · Panvalet (1970s) · Perforce Helix (1995) · SCLM (1980s?) · Software Change Manager (1970s) · StarTeam (1995) · Surround SCM (2002) · Synergy (1990) · Team Concert (2008) · Vault (2003) · Visual SourceSafe (1994)
Distributed	Free/open-source	ArX (2003) · BitKeeper (2000) · Codeville (2005) · Darcs (2002) · DCVS (2002) · Fossil (2007) · Git (2005) · GNU arch (2001) · GNU Bazaar (2005) · Mercurial (2005) · Monotone (2003)
	Proprietary	Azure DevOps (Server (via Git) (2013) · Services (via Git) (2014)) · TeamWare (1992) · Code Co-op (1997) · Plastic SCM (2006)
Concepts	Baseline · Branch · Changeset · Commit · Data comparison · Delta compression · Fork (Gated commit) · Interleaved deltas · Merge · Monorepo · Repository · Tag · Trunk	



Các loại công cụ quản lý phiên bản

- Tập trung
 - client-server
/centralized version control system
 - Ví dụ: CVS (Concurrent Versions System) và SVN (SubVersion)



Các loại công cụ quản lý phiên bản

Apache Subversion



Original author(s) CollabNet

Developer(s) Apache Software Foundation

Initial release 20 October 2000; 20 years ago

Stable release(s) [±]

1.10.x 1.10.7 / 10 February 2021; 2 months ago^[1]

1.14.x 1.14.1 / 10 February 2021; 2 months ago^[1]

Concurrent Versions System

Developer(s) The CVS Team [↗](#)

Initial release November 19, 1990; 30 years ago

Stable release 1.11.23 / May 8, 2008; 12 years ago

Preview release 1.12.13 / July 26, 2006; 14 years ago

Repository savannah.nongnu.org/projects/cvs [↗](#)

Written in C

Operating system Unix-like, Windows

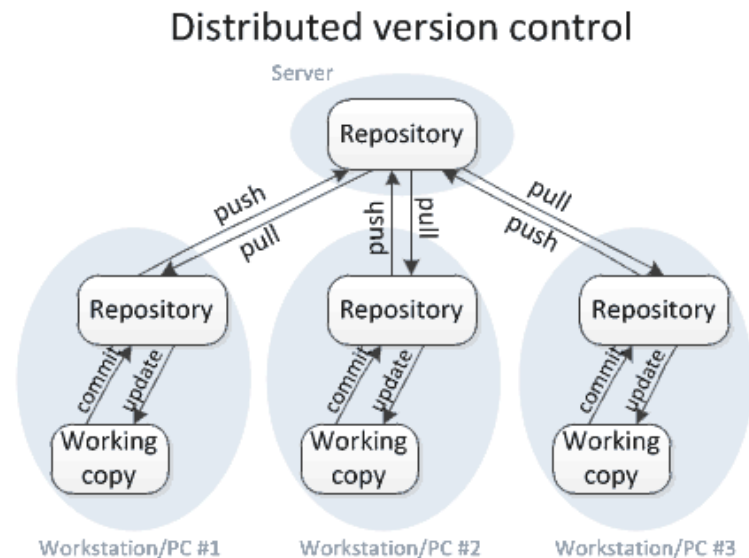
Type Revision control

License GNU General Public License

Website cvs.nongnu.org [↗](#)

Các loại công cụ quản lý phiên bản

- Phân tán
 - Distributed version control system
 - Mỗi người dùng có kho lưu trữ của riêng mình và bản sao làm việc
 - Ví dụ: Git, Mercurial



So sánh hai loại quản lý phiên bản

- Quản lý tập trung
 - Tránh conflict hoặc out of update
- Quản lý phân tán
 - An toàn hơn so với quản lý tập trung
 - Khi server bị down, các thành viên vẫn có thể làm việc offline, họ vẫn có thể commit và update trên local của họ
 - Khi server hoạt động trở lại, họ có thể cập nhật tất cả lên lại server

GỠ LỖI

Gỡ lỗi

- Debug (gỡ lỗi) là một kĩ năng nền tảng của lập trình viên
 - Loại bỏ lỗi (error) khỏi chương trình
 - Hiểu rõ hơn sự thực thi của chương trình

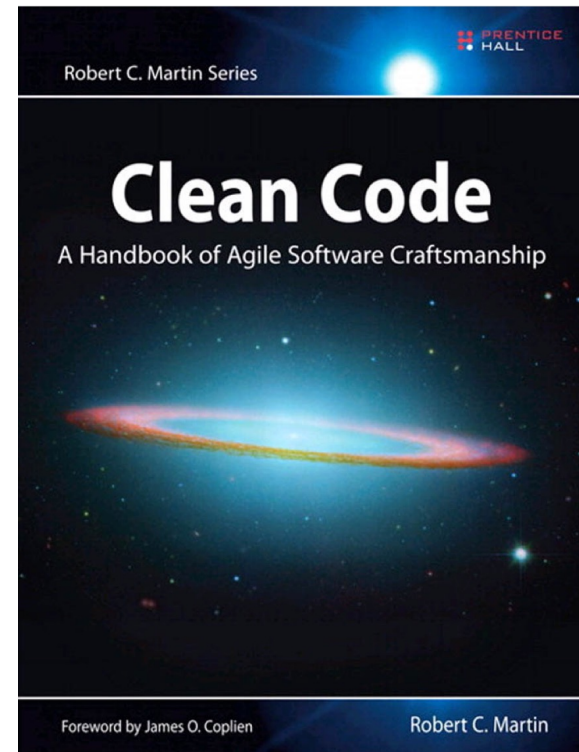
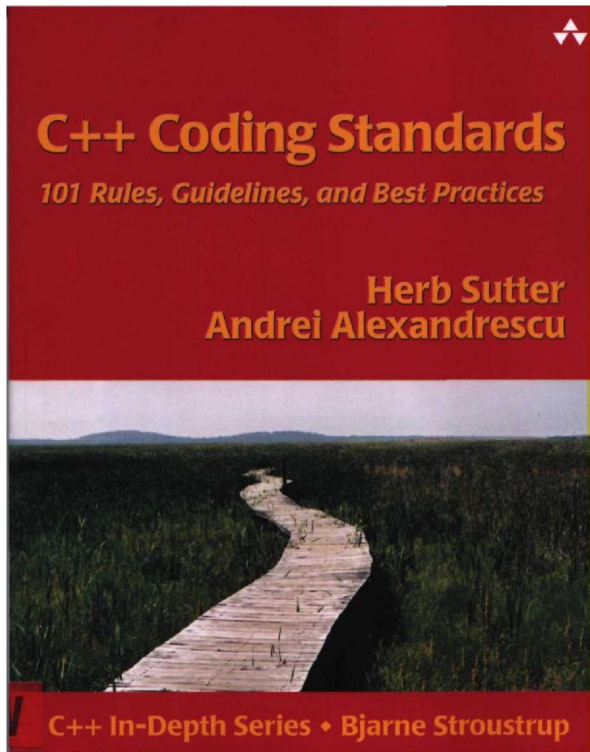
Các phương pháp gỡ lỗi

- Debugging Tool
 - Dùng công cụ để debug
 - Microsoft Visual Studio Debugger , GNU Debugger
- Printlining
 - Thêm dòng lệnh để in ra những thông tin cần theo dõi
- Logging
 - Ghi lại những thông tin sau khi chương trình thực thi

Tài liệu tham khảo

- Robert C. Martin. 2008. Clean Code: A Handbook of Agile Software Craftsmanship (1st. ed.). Prentice Hall PTR, USA.

Some interesting books



ANY QUESTION?